
Defense Study Guide

How to master the thesis — theory, code, methodology, and results — before the oral defense.

Evaluation of Metaheuristic Algorithms for Energy Optimisation in Scheduling and Routing

Author: Christian Wu

DTU Compute — M.Sc. Computer Science and Engineering

July 2026

Who this is for: you, the author, preparing to defend this thesis in front of professors who will ask specific, in-depth questions about the theory, the code, the experimental design, and the results. You already understand the broad picture; this guide takes you from "I know roughly what we did" to "I can answer any question about any formula, any file, and any number in the report."

How to use it: work through the phases **in order**. Each phase tells you exactly what to read, in what order, why, and how to check that you actually understood it. Don't skip the self-test checkpoints — they are the whole point. Reading passively feels like progress but doesn't survive a professor's follow-up question. Explaining out loud does.

Table of Contents

- The one-paragraph summary you must own
- Time estimates — how long until you're ready?
- Phase 0 — Orientation (half a day)
- Phase 1 — The two problems, cold (1–2 days)
- Phase 2 — The algorithms: theory + code together (4–6 days)
- Phase 3 — The experimental methodology (2–3 days)
- Phase 4 — The results: own every number (2–3 days)
- Phase 5 — Weak points and likely professor questions (2 days)
- Phase 6 — Rehearsal (2 days)
- How to read a code file (method)
- The master question bank

The one-paragraph summary you must own

If you can say this fluently, everything else hangs off it:

The thesis compares metaheuristic families on two structurally different energy-related combinatorial problems: an assignment problem (cloud task scheduling: 50 tasks → 10 servers, integer vector encoding) and a permutation problem (single-vehicle EV routing: 75 customers + charging stops in San Francisco, route encoding). "Structurally neutral" algorithms (SA, GA) run on both; "structure-specific" ones run where their model fits (UMDA on the assignment problem, ACO on the routing problem, plus a Memetic extension of GA on routing). Everything gets the same 150,000-evaluation budget, 10 seeds, tuned hyperparameters, and Deb-calibrated objective weights, so differences in results reflect the algorithms — not budget, tuning, or unit-scale artifacts. The three research questions (Ch. 1) ask whether structure-specific methods beat neutral ones, whether the neutral ones transfer across problems, and whether the rankings survive hyperparameter sensitivity analysis. Success is judged against criteria C1–C4 (Ch. 5): beats baseline, scales, small optimality gap, robust tuning.

Everything a professor asks is a zoom-in on one clause of that paragraph.

Time estimates — how long until you're ready?

Honest estimates, assuming focused study (no phone, notes open, code editor open):

For you (co-wrote the thesis, know the broad structure already)

Goal	Focused hours	Calendar time
Defense-ready (can answer ~90% of in-depth questions, knows where every formula lives, can whiteboard every algorithm)	40–60 hours	~2 weeks at 4 h/day, or ~3 weeks at 3 h/day
Comfortable + confident (adds full question-bank rehearsal, mock defense, second pass over weak spots)	60–80 hours	3–4 weeks at 3–4 h/day

The phases below add up to roughly 13–18 study days. If your defense is closer than that, do Phases 0, 1, 2, 5, 6 and compress Phases 3–4 into reading Chapters 7–8 carefully once — those are the highest-value phases.

For someone starting from scratch (general CS background, no metaheuristics)

Goal	Focused hours	Calendar time
Understand the thesis and follow the code	~80–120 hours	3–4 weeks part-time
"Thesis-expert" — could defend it themselves	~150–250 hours	5–8 weeks part-time, or 3–5 weeks full-time
General expert in metaheuristics as a research field	6–12+ months	Reading Talbi's <i>Metaheuristics</i> textbook, the original papers (Kirkpatrick 1983, Holland/Goldberg, Mühlenbein UMDA, Dorigo ACO, Deb 2000/2001), and implementing things yourself

Key realism check: you do *not* need to be a general expert in metaheuristics to defend. You need to be *the* expert on **this thesis** — its choices, its code, its numbers, and its limitations. That is a bounded, achievable goal, and it's what this guide targets. Nobody in the room will know this codebase better than you after Phase 4.

Phase 0 — Orientation (half a day)

Goal: see the whole machine run once, end to end, before studying any part of it.

1. **Read the thesis Summary and Chapter 1** (`Introduction.tex`). Write down, on paper, in your own words:
 - The three research questions (Section 1.3). Memorize them — every defense answer should eventually connect back to one of them.
 - The contributions list.
2. **Run both experiments small** so you have fresh outputs to poke at: `bash uv run run.py cloud --seeds 3`
`PYTHONPATH=EV_routing python EV_routing/main.py`
3. **Read the generated summaries:** `Cloud_scheduling/results/summary.md` and `EV_routing/results/sf_75/summary.md`. These are the "answers" the whole pipeline produces — everything you study afterwards explains *how* these numbers came to be.
4. **Skim the `README.md` top to bottom** (don't study it yet — just build the map). Note that it contains a `Thesis Formula Verification` table and a `Limitations` section — you will come back to both.

✓ **Checkpoint:** without looking, say out loud: the three RQs, the two problems, the five algorithm names per problem, and the evaluation budget. If you can't, re-read Chapter 1 — do not proceed with a fuzzy top level.

Phase 1 — The two problems, cold (1–2 days)

Goal: be able to write both objective functions and both solution representations on a whiteboard, from memory, and explain every symbol. This is the single most likely thing a professor asks you to do.

Day 1 morning — Cloud problem

1. Read **Chapter 3, Section 3.1** (`Problem Specification.tex`).
2. Immediately cross-check against code — theory and code must fuse in your head: -
`Cloud_scheduling/tools/objective.py` — find `evaluate_schedule()` . Match every term to the thesis formula: $F(X) = w_e \cdot \tilde{E} + w_l \cdot \tilde{L} + \lambda_{\text{cpu}} \cdot (\text{CPU violation}) + \lambda_{\text{mem}} \cdot (\text{mem violation})$. - The energy model $E(X) = \sum \text{idle} \cdot y + \sum \eta \cdot e$ — find the two sums in the code. - The congestion latency $\hat{l}_i = l_i \cdot (1 + \gamma \cdot U/C)$ and priority weights $\omega = 1/2/4$. - Use the **Thesis Formula Verification table** in the README — it maps each formula to exact lines.
3. Understand the **representation**: integer vector `x[i]` = server of task `i` , and why this implicitly enforces the one-hot constraint $\sum_j x_{ij} = 1$.

Day 1 afternoon — EV problem

1. Read **Chapter 3, Section 3.2** and **Section 3.3** (structural differences — this section is the intellectual core of the whole thesis: assignment vs permutation structure).
2. Cross-check against code: - `EV_routing/tools/objective.py` — `evaluate_route()` : distance, time (travel + charging), energy, charging cost, battery-violation penalty, infeasible-visit penalty. - `EV_routing/tools/battery.py` — `EVParameters` : 20 kWh capacity, 0.50 kWh/km base, grade factor 3.0, speed exponent 2.0. Be able to explain *how an arc's energy is computed* (distance $\times$ base rate, scaled by grade and speed terms). - `EV_routing/tools/data_loader.py` — how the energy matrix is built at load time from the distance/duration/elevation CSVs.
3. Understand the **representation**: `["DEPOT", "C001", "EVS04656", ..., "DEPOT"]` — customers exactly once, stations optional and repeatable, recharge-to-full.

Day 2 — Chapter 2 + Chapter 5

1. Read **Chapter 2** (`Related work.tex`) once, for the *research gap* argument: what has been done, and why "same-budget cross-paradigm comparison on two structurally different problems" is the gap this thesis fills. You don't need to memorize citations — you need the *shape* of the argument.
2. Read **Chapter 5** (`Problem Analysis and Success Criteria.tex`): - Search-space sizes (10^{50} assignments; $75!$ station-insertions routes) — be able to reproduce the argument for why exhaustive search is hopeless. - **Memorize C1–C4**: C1 beats baseline (statistically), C2 scales, C3 small optimality gap, C4 robust tuning. The Conclusion is structured around these — professors love asking "so did algorithm X meet your own criteria?"

✅ **Checkpoint:** whiteboard test. Blank page, write both objective functions with every symbol defined, both representations, and C1–C4. Then explain (out loud) why the cloud problem is "assignment-shaped" and EVRP is "permutation-shaped", and why that difference is exactly what RQ1 exploits.

Phase 2 — The algorithms: theory + code together (4–6 days)

Goal: for every algorithm, be able to (a) whiteboard its pseudocode, (b) explain every design choice ("why tournament selection?", "why Laplace smoothing?"), (c) point to the file and function implementing it, and (d) explain how it behaves in *your* results.

Method for each algorithm (repeat ~7 times, ½ day each): 1. Read its section in **Chapter 4** (*Metaheuristic Optimisation Methods.tex*). 2. Read its pseudocode in **Appendix A** (*Pseudocode.tex*). 3. Read its implementation top to bottom (files below). 4. Read its paragraph in **Chapter 6** (*Implementation.tex*, Sections 6.4/6.5) — this is where thesis text and code explicitly meet. 5. Close everything and write the pseudocode from memory. Check. Repeat until clean.

Suggested order (each builds on the previous)

#	Algorithm	Code	Must be able to explain
1	Greedy baselines	<code>baselines.py</code> , <code>greedy.py</code>	Cloud greedy is Best-Fit Decreasing (sort by CPU desc). EV greedy is nearest-neighbour with proactive charging at 50% battery. Why baselines matter: C1 is measured against them.
2	Simulated Annealing	<code>Cloud SA</code> , <code>EV SA</code>	Metropolis rule $\exp(-\Delta F/T)$; geometric cooling $T \cdot \alpha$; auto-T_0 calibration (~80% initial acceptance, from 400 probe moves — know why this matters when the objective is normalised); reheating (patience 300, factor $0.4 \cdot T_0$); the 5 cloud move operators / shared EV operators.
3	Genetic Algorithm	<code>Cloud GA</code> , <code>EV GA</code>	Tournament selection $k=3$; uniform crossover on the cloud (per-gene, works because genes are independent positions) vs OX order crossover on EVRP (uniform would break the permutation — this contrast is a core thesis point!); mutation $p=1/n$; elitism 2.
4	UMDA	<code>umda.py</code>	Estimation-of-Distribution idea: learn $P[\text{task}][\text{server}]$ from top-50% truncation selection, sample new population. Laplace smoothing $\alpha=0.1$ (why: no server probability ever hits 0). Shannon entropy as a convergence diagnostic ($\log_2 10 \approx 3.32$ bits = uniform). The univariate independence assumption is its known theoretical weakness — be ready for that question. Why UMDA fits assignment but not permutation (sampling independent marginals can't produce valid permutations without repair).
5	ACO (Max–Min Ant System)	<code>ant_colony.py</code>	Construction via pheromone τ and heuristic η with battery-aware feasibility; pheromone bounds $[\tau_{\min}, \tau_{\max}]$ (why MMAS and not plain AS: prevents premature convergence); evaporation; why ACO fits routing (pheromone lives on <i>arcs</i> , which is exactly the permutation structure) and pairs with EVRP the way UMDA pairs with cloud.
6	Memetic Algorithm	Same file as EV GA (<code>local_search_iters > 0</code>)	GA + first-improvement local search on offspring; the exploration/exploitation argument; what it costs (local search consumes evaluations from the same budget).

#	Algorithm	Code	Must be able to explain
7	Branch & Bound	<code>branch_and_bound.py</code>	Depth-first tree over task assignments, prune when lower bound $\geq$ best. Critical nuance: in this thesis B&B is a lower-bound reference on a small instance (n=20, m=4), NOT a proven-exact solver at full scale — the LB is loose, and the reported "gap" must be interpreted carefully (see README limitations). This is a known trap question.

Also in this phase

- Read the **shared tools** once each, lightly: `neighborhoods.py (cloud)` (5 operators: reassign/swap/rescue/consolidate/spread) and `neighborhoods.py (EV)` (8 operators: customer swap/relocate/2-opt + station insert/remove/replace/move + battery repair). Be able to name all operators and say which algorithms use them.
- `initial_solution.py (EV)` — `build_ev_feasible_solution()` and the station-repair logic: how does an infeasible route become feasible?

✅ **Checkpoint per algorithm:** the "why" quiz. Not "what does it do" but: Why geometric cooling and not linear? Why $k=3$ and not $k=7$? Why uniform crossover here but OX there? Why does UMDA need smoothing? Why MMAS bounds? Why is the MA's local search *first-improvement* and not best-improvement? If you can't answer a "why", the answer is usually in Chapter 4, Chapter 6, or the README algorithm sections — go find it *now*, while the question is fresh.

Phase 3 — The experimental methodology (2–3 days)

Goal: defend the *fairness* of the comparison. This is where thesis defenses are won or lost: professors probe methodology harder than algorithms.

1. Read **Chapter 7** (`Experimental Setup.tex`) slowly. The pillars: - **Equal evaluation budget:** 150,000 `evaluate()` calls per algorithm per run (Ch. 6.6 explains the mechanism). Know how the budget maps to each algorithm: SA 3,000 temp steps $\times$ 50 iters; GA/UMDA 100 pop $\times$ 1,500 generations. Know the small asymmetry: SA's T_0 probe adds ~ 400 evals ($\sim 0.3\%$) — it's measured and reported, not hidden. - **Weight calibration (Deb 2001 sample normalisation + Deb 2000 penalty rule):** each objective term normalised to contribute ≈ 1 in expectation over a feasible sample pool; penalties $\lambda = 100 \times \max$ feasible objective so any infeasible solution is dominated by every feasible one. Read the README's `Normalisation and Penalty Calibration` section — it's the most complete write-up — then the code: `Cloud objective.py` (`compute_sample_normalization`) and `EV calibrate_weights.py` . **Be able to explain why weights (1,1) would be meaningless without this** (Watts vs milliseconds unit mismatch would silently set the real preference). - **Hyperparameter tuning protocol:** tune once, freeze, then evaluate. Cloud: grid search (`--tune`). EV: random search, 30 trials $\times$ 2 seeds $\times$ 50k evals (`tune.py` $\rightarrow$ `results/sf_75/params.json`). Why you must NOT re-tune per experiment (tuning on the test instance biases the comparison). - **Statistics:** 10 seeds; **Wilcoxon signed-rank tests** (`statistics.py`). Know what the test does (paired, non-parametric, no normality assumption — that's why not a t-test), what "significant at $\alpha=0.05$ " means, and the honest caveat: 10 seeds is a small sample, so results are exploratory. Detail worth knowing: the appendix tables use the *exact* Wilcoxon distribution; the run logs use the normal *approximation* — tiny p-value differences between the two are expected. - **Focus modes:** cloud balanced/performance/eco (weights + γ); EV balanced/eco/time. Cross-mode objective values are NOT comparable (mode-dependent λ) — within-mode comparisons are.
2. Read **Chapter 6** in full now (you've read pieces): architecture, tools, equal-budget section.
3. Skim the config artifacts so you can name where every number lives: `Cloud config.yaml` , `EV_routing/results/sf_75/params.json` (thesis Table 8.3), `EV_routing/results/sf_75/weights.json` (thesis Table 7.1), `Cloud_scheduling/results/run_manifest.yaml` (full reproducibility snapshot).
4. Understand the **data honestly**: both Kaggle input datasets are **synthetic** (the thesis says so — never call them "real"), the cloud server pool is hand-specified, BUT the EV distances/durations come from real **OSRM road-network queries** and elevations from **SRTM** — so the SF *road network* is real even though customer demand is synthetic. Get this distinction crisp; it is a guaranteed question area.

✓ **Checkpoint:** explain out loud, in under 3 minutes each: (a) how you made the comparison fair (budget, tuning, calibration, seeds, tests), (b) the full journey of one number — from a CSV row to a cell in a results table — naming every file it passes through.

Phase 4 — The results: own every number (2–3 days)

Goal: for every table and figure in Chapter 8, you can say what it shows, why it looks like that, and what conclusion it supports.

1. Go through **Chapter 8** (`Results and Evaluation.tex`) figure by figure, table by table. For each one write a one-sentence answer to: *"What would I say if the professor put this on the screen and said 'explain'?"* - Convergence plots: why does UMDA's curve flatten where it does (model entropy collapse)? Why does SA improve late (reheats)? - Box plots: which algorithm has highest variance and why? - Scalability: know the fixed-budget story — at $n \geq 200$ tasks, SA and UMDA collapse toward greedy while GA keeps improving. Why: SA is a single trajectory from the greedy start; UMDA's model has $n \times m$ parameters but only ~ 50 selected samples per generation. This is a *finding*, not a bug — population diversity scales better at fixed budget. - Sensitivity: which parameters were robust, which sensitive; how this answers RQ3 / criterion C4. Cloud detail: the SA cooling-rate sweep equalises the *schedule* (auto-scales `max_temp_steps` per α) so every α gets the same budget — know this if asked "did faster cooling just get fewer steps?" (No.) - Optimality gaps: metaheuristics vs the B&B bound on the 20-task instance (C3).
2. Read **Chapter 9** (`Comparative Discussion.tex`) and **Chapter 10** (`Conclusion.tex`) together. Build one page of notes: **RQ** $\rightarrow$ **evidence** $\rightarrow$ **answer**, and **C1–C4** $\rightarrow$ **verdict per algorithm**. Note the structure is intentional: Ch. 9 synthesises across problems, Ch. 10 answers the RQs — know which chapter says what.
3. Check figures against your Phase-0 run outputs in `Cloud_scheduling/results/` and `EV_routing/results/sf_75/` — the pipeline that made the thesis figures is the one you ran.
4. Read **Appendix B** (Additional Experimental Material) once so nothing in your own document surprises you.

✅ **Checkpoint:** the RQ drill. For each of RQ1/RQ2/RQ3, give a 60-second spoken answer that names the specific evidence (which table, which test, which figure). Then the reverse drill: pick any figure at random and connect it back to an RQ.

Phase 5 — Weak points and likely professor questions (2 days)

Goal: never be surprised. Every thesis has soft spots; yours are *documented* — which is a strength if you own them, and a disaster if the professor finds them first.

1. Study the README's **Limitations and Threats to Validity** section like an exam syllabus, plus Chapter 10's Limitations section. The big ones: - **EV energy model simplifications** — acknowledged in the thesis, deliberately not "fixed" (changing them would invalidate all run results). Be ready to say what's simplified (linear consumption + grade/speed factors, recharge-to-full, no queueing at stations) and why the *comparison between algorithms* is still valid (all algorithms face the same model — internal validity holds). - **Synthetic data** — internal vs external validity argument (Phase 3, item 4). - **B&B gap interpretation** — lower-bound reference, loose LB at tight constraints; not evidence the metaheuristics are far from optimal. - **10 seeds** — exploratory statistics, honest about it; 30+ seeds is future work. - **Single instance per setting** — "would rankings hold on a different 50-task draw?" Partially addressed by the scalability axes; full instance sweep out of scope. - **UMDA independence assumption** — scope choice (representative EDA), BMMA/BOA would model dependencies but are out of scope. - **Soft constraints** — the Deb 2000 defense: penalties calibrated so infeasible never beats feasible; smooth landscape is *why* penalties instead of repair.
2. Work through the **Master question bank** below. For each question: answer out loud first, *then* check the pointer. Mark the ones you fumbled and redo them the next day.
3. Prepare your three "steer-the-conversation" strengths — things you *want* to be asked about because you shine: e.g. the calibration methodology, the uniform-vs-OX crossover contrast, the fixed-budget scalability finding.

Phase 6 — Rehearsal (2 days)

1. **The chapter walk:** explain each of the 10 chapters out loud in ≤ 2 minutes each, no notes. Record yourself once; listening back is uncomfortable and extremely effective.
2. **Whiteboard drills** (paper is fine): both objective functions; SA acceptance rule + cooling; UMDA update equation with smoothing; the OX crossover on a 6-customer example; the Deb penalty rule. These are the things professors ask you to *write*, not just say.
3. **Mock defense:** give a friend/partner the question bank and have them fire random questions for 45 minutes. If nobody's available, shuffle the bank and answer to an empty room — out loud, full sentences. (You can also open this repo in Claude Code and ask it to play examiner using this guide's question bank — it knows the codebase.)
4. **The night before:** re-read only Chapter 1, Chapter 10, and your RQ→evidence page. Nothing new. Sleep.

How to read a code file (method)

Same recipe every time — don't just scroll:

1. **Entry point first.** Find the function the thesis names (e.g. `simulated_annealing(...)`). Read its signature and docstring: inputs, outputs.
2. **Trace one iteration by hand.** Take a tiny imaginary instance (3 tasks, 2 servers / 4 customers) and walk one loop iteration on paper: what's the current solution, what move is proposed, what does `evaluate` return, what's accepted.
3. **Map each block to a thesis equation.** Every non-trivial line should correspond to something in Ch. 3/4/7 or the README verification table. If a line corresponds to nothing, understand it anyway — "what's this line for?" is a legitimate defense question.
4. **Ask "what breaks if I delete this?"** for each mechanism (elitism, smoothing, reheating, repair). The answer *is* the justification for the mechanism.

For the deep file-by-file tour of the cloud code there is already a dedicated document:

`Cloud_scheduling/BEGINNERS_GUIDE.md`. Use it during Phase 2. The EV code follows the same architecture (same tool-file names, same experiment harness pattern), so the cloud guide's reading order transfers almost one-to-one.

The master question bank

Answer out loud before checking the pointer. ✓ the ones you got; redo the rest tomorrow.

Problem formulation

1. Write the cloud objective $F(X)$ and define every symbol. → Ch. 3.1, `README`
2. Why is the one-hot assignment constraint not in the code? → integer encoding enforces it implicitly.
3. Write the EV objective. Which terms are real costs and which are penalties? → Ch. 3.2, `EV_routing/tools/objective.py`
4. How is arc energy computed, exactly? What do grade factor 3.0 and speed exponent 2.0 do? → `battery.py`, `data_loader.py`, Ch. 3.2
5. Why can charging stations be visited multiple times but customers exactly once?
6. What are the search-space sizes, roughly, and why does that justify metaheuristics? → Ch. 5.1
7. Why soft penalties instead of hard constraints with repair? → Deb 2000; smooth explorable landscape; `README` limitations.

Algorithms

8. Derive/justify the Metropolis acceptance probability. What happens as $T \rightarrow 0$ and $T \rightarrow \infty$?
9. How is T_0 chosen and why does auto-calibration matter with a normalised objective? → `estimate_initial_temperature()`, ~80% initial acceptance.
10. Why geometric cooling? What does reheating do, and when does it trigger?
11. Why uniform crossover for cloud but OX for EVRP? What would uniform crossover do to a permutation? ← **core thesis question**
12. Why mutation probability $1/n$? What's the expected number of mutations per child?
13. Explain UMDA's model update with Laplace smoothing. What goes wrong with $\alpha=0$?
14. What does UMDA's model entropy tell you? What are its max and min values here?
15. Why can't UMDA (as implemented) be applied to the EVRP directly? ← **the representation-alignment argument, RQ1**
16. Explain MMAS: pheromone bounds, evaporation, why not plain Ant System.
17. How does the ACO construction handle battery feasibility?
18. What exactly does the Memetic Algorithm add over GA, and what does it pay for it?
19. How does B&B prune? Why is its reported gap on tight instances misleading? → `README` limitations.
20. Name the 5 cloud move operators and the 8 EV operators. Which algorithms share them?

Methodology

21. Justify "150,000 evaluations for everyone" as the fairness criterion — why evaluations and not wall-clock time?
22. Walk me through Deb 2001 sample normalisation step by step. Where do E_{ref} and L_{ref} come from?
23. Why $\lambda = 100 \times F_{max}(\text{feasible})$? What property does that guarantee?
24. What were the tuning protocols (cloud vs EV) and why tune-once-freeze?
25. Why Wilcoxon signed-rank and not a t-test? What does a p-value < 0.05 mean here, precisely?
26. How many seeds, and is that enough? (Honest answer: exploratory, documented limitation.)
27. What in the data is synthetic and what is real? ← **guaranteed question; get the OSRM/SRTM nuance right**
28. Can you compare objective values between eco and balanced mode? (No — mode-dependent λ and weights; why?)

Results

29. Which algorithm won on each problem, and by how much over the baseline?
30. Did each algorithm meet C1–C4? Go criterion by criterion. → Ch. 10 + your Phase-4 notes
31. Why do SA and UMDA collapse toward greedy at $n \geq 200$ tasks while GA doesn't? ← **the fixed-budget finding**
32. In the SA cooling sweep, did small α win just because of a different schedule length? (No — schedule-equalised; explain.)
33. Which hyperparameters turned out sensitive, which robust? What does that mean for C4/RQ3?
34. What do the optimality gaps against B&B actually establish, and on what instance size?
35. Answer RQ1. Answer RQ2. Answer RQ3. With evidence, in ≤ 60 seconds each.

Big picture

36. What is the research gap and what is the thesis's main contribution, in two sentences?
37. If you had three more months, what would you do first and why? → Ch. 10 Future Work — have a genuine, ranked answer.
38. What would you do differently if you started over? (Prepare an honest, non-fatal answer — e.g. more seeds, an instance sweep, a richer EV energy model validated against real consumption data.)

Good luck. Remember: after Phases 0–4 you will know this system better than anyone who will ever question you about it. The defense is your home field.